

시큐어코딩



SW개발보안 이해

01 현대 애플리케이션 보안 과제

02 사고사례를 통한 시큐어코딩 이해



학습 목표

- ✓ 현대 애플리케이션의 보안과제를 이해한다
- ✓ 현대 애플리케이션 보안과제를 해결하기 위한 방안을 이해한다.
- ✓ 사고사례를 통한 시큐어코딩을 이해한다.



학습 내용

1. 현대 애플리케이션 보안 과제
2. 사고사례를 통한 시큐어코딩 이해



SW개발보안 이해

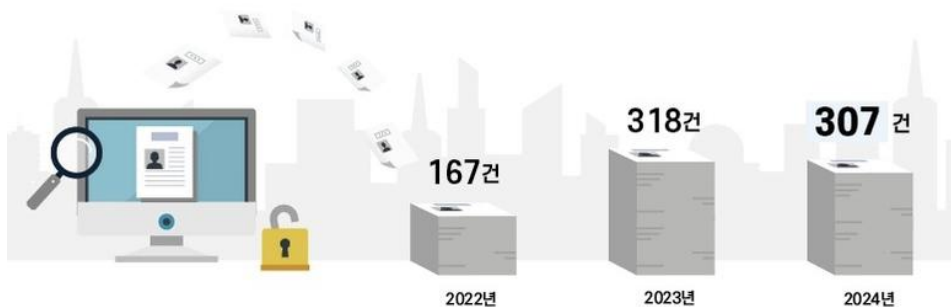
01 현대 애플리케이션 보안 과제

02 사고사례를 통한 시큐어코딩 이해

데이터 유출현황 및 침해발생 원인

최근 3년간 개인정보 유출 사고

최근 3년간 개인정보 유출 신고 현황

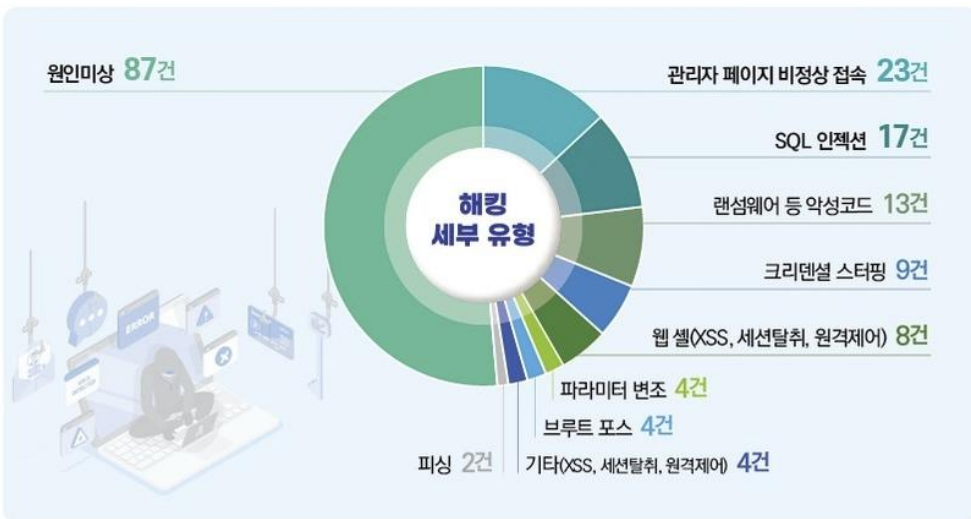


출처: 개인정보보호위원회 제공

2024년 개인정보 유출 사고 원인



원인미상 87건



데이터 유출현황 및 침해발생 원인

2020년 한국 기업데이터 유출현황과 데이터 침해 발생원인

국내 SW, 잘 알려진 보안 취약점도 대응 못해

| xss·SQL 등 익숙한 유형이 다수... "SW 개발보안 인식 부족 방증"

컴퓨팅 | 입력 : 2021/02/19 11:24 수정 : 2021/02/19 11:24



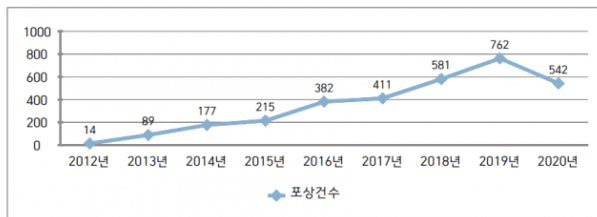
김윤희 기자 | ✉ 기자 페이지 구독 📖 기자의 다른기사 보기



국내 소프트웨어(SW)에서 발견되는 취약점 중 대부분이 이미 흔히 일
형의 취약점인 것으로 나타났다.

한국인터넷진흥원(KISA)은 이같은 내용을 담은 '보안 취약점 신고포상
해 알아본 놓치기 쉬운 취약점 사례별 대응 방안' 보고서를 지난 16일
다.

KISA는 지난 2012년부터 보안 취약점 신고 포상제를 운영하고 있다.
부터는 웹사이트 취약점 발굴을 위한 모의 해킹 대회 '핵 더 챌린지'도
포상이 지급된 취약점 건수는 작년까지 총 3천173건이다. 2019년까지 매해 신고 건수가 늘다 작년에는 두서너
다소 감소했다.



[그림 1] 연도별 보안 취약점 포상건수

연도별 보안 취약점 포상 건수(출처=KISA)

OECD "소스코드는 곧 취약점...쉬쉬해선 안돼"

| 취약점 대응 체계 활성화 필요성 강조

컴퓨팅 | 입력 : 2021/03/17 17:03



김윤희 기자 | ✉ 기자 페이지 구독 📖 기자의 다른기사 보기



"취약점은 디지털 생활의 일부다. 모든 취약점을 제거하는 것은 불가능하다.
그보다는 취약점 조치를 개선해 보안 위험을 줄이고 디지털 전환 시대의 신뢰
를 도모해야 한다."

보안 취약점이 드러나는 것을 사업 리스크나 잘못으로 받아들이기보다, 일상
적인 현상으로 이해하고 관련 제보 체계 및 지속적인 보안 업데이트를 정착시
켜야 한다는 경제협력개발기구(OECD) 보고서가 나와 눈길을 끈다.

한국인터넷진흥원(KISA)은 OECD가 이같은 내용을 담아 지난달 발표한 '취약점 조치의 장려(Encouraging
vulnerability treatment)' 보고서 번역본을 지난 16일 공유했다.

■ 취약점 알려줘도 고소·고발로 받아치는 경우 허다

현대 애플리케이션의 보안 과제

봇관리

인터넷 트래픽의 52%는
봇에 의한 통신

- 그중 절반은 악성 봇
- 기업의 79%는 악성봇과 정상봇을 구분하지 못함
- 악성봇은 네트워크 트래픽 낭비, 도스/디도스 공격, 중요정보/개인정보 수집, 캡차우회 지능적인 봇 공격수행, 크리덴셜 스테핑을 통한 개인정보 수집에 이용됨

API 보안

API로 연결된 다양한 환경

- IoT, 클라우드, 웹, 모바일 등 다양한 환경에서 애플리케이션이 사용되며, API로 연결
- API를 통해 서로의 리소스와 정보 공유
- API 취약점을 이용해 각종 인젝션, 파라미터 조작, 공격사이트로 리다이렉트, 봇 공격 등을 발생시킴

서비스 거부(DoS)

여전히 서비스 장애를 일으키는 효과적인 공격

- 애플리케이션 계층 DoS 공격은 매우 오래된 공격 방식이지만 서비스를 장애를 일으키는데 여전히 효과적
- HTTP/S 플러드, 슬로우로리스, LOIC, 버퍼오버플로, 브루트 포스 공격
- IoT봇넷에 의해 구동되는 애플리케이션 계층 공격은 선호되는 디도스 공격 벡터

지속적인 보안

DevOps개발환경에서 민첩성을 위해 희생 되는 보안

- 신속한 개발과 롤아웃 방법론은 애플리케이션을 지속적으로 수정하고 업데이트 하지만 보안을 고려하지 못하는 경우가 발생
- 개발완료된 서비스의 취약점을 개선하고 보안을 주입하는 것은 더 어려운 일
- AI기술을 활용한 리소스 매핑, 가능한 위협 분석, 실시간 보안 정책을 생성하고 최적화

출처: 라드웨어 애플리케이션 보호 위한 4가지 도전과제 기술백서

애플리케이션 보안 과제는 어떻게 해결해야 할까?

SW개발보안 프로세스

DevSecOps

보안내재화

Security By Design

Zero Trust

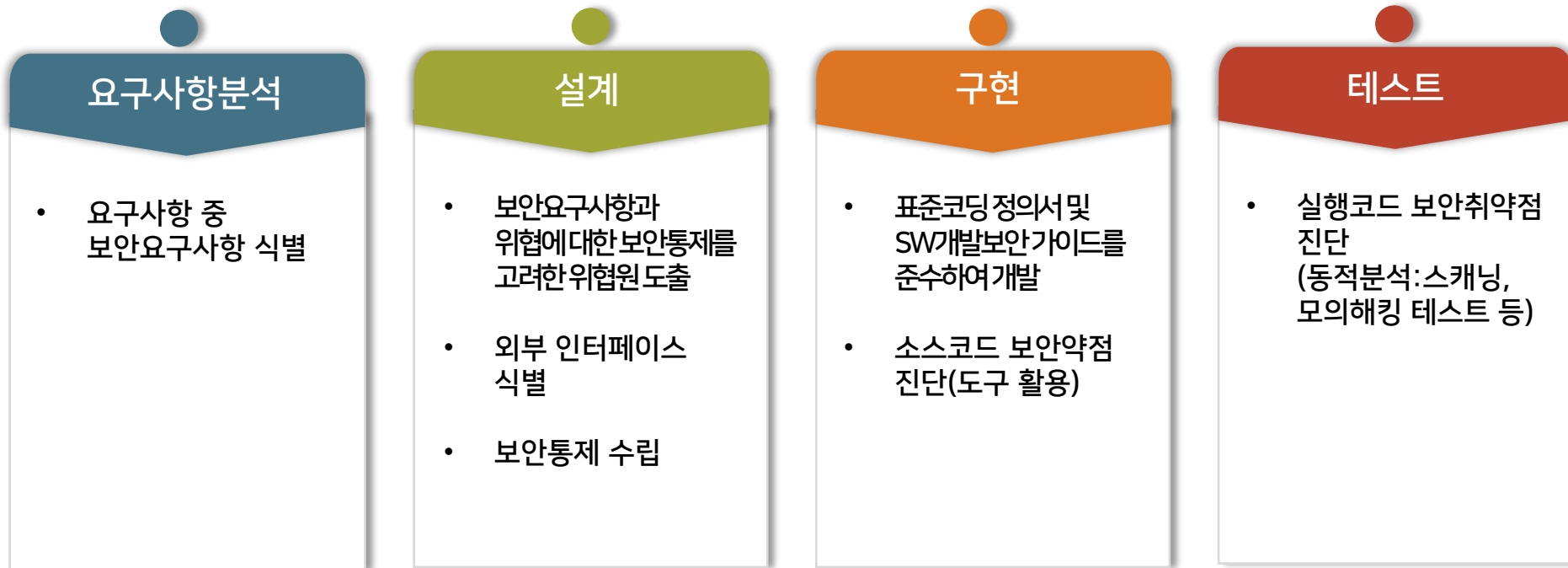
애플리케이션 보안(1)

SW개발보안 프로세스 DevSecOps

애플리케이션 수명주기 따른 반복·지속적 보안 제어 필수

SW개발보안 프로세스

SW개발과정에서 개발자의 실수, 논리적 오류 등으로 인해 발생할 수 있는 보안약점들을 최소화하여 안전한 SW를 개발하기 위해 소프트웨어 개발 각 단계별 적절한 보안활동 수행

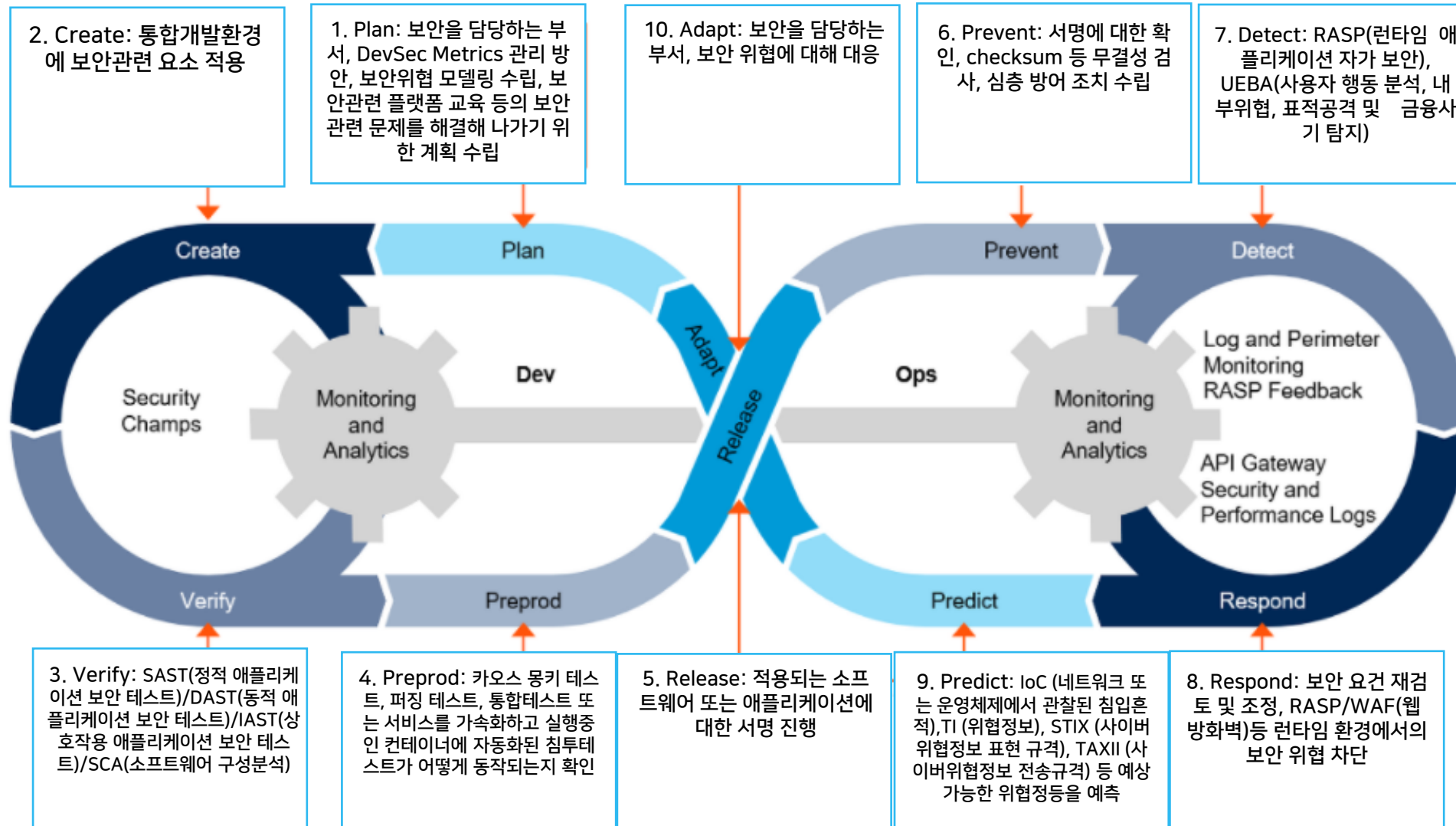


※ 현재는 SW 개발과정 중 소스코드 구현단계를 중심으로 SW 개발보안 적용

※ 보안취약점(Vulnerability) : 해킹 등 실제 보안사고에 이용되는 SW 보안약점

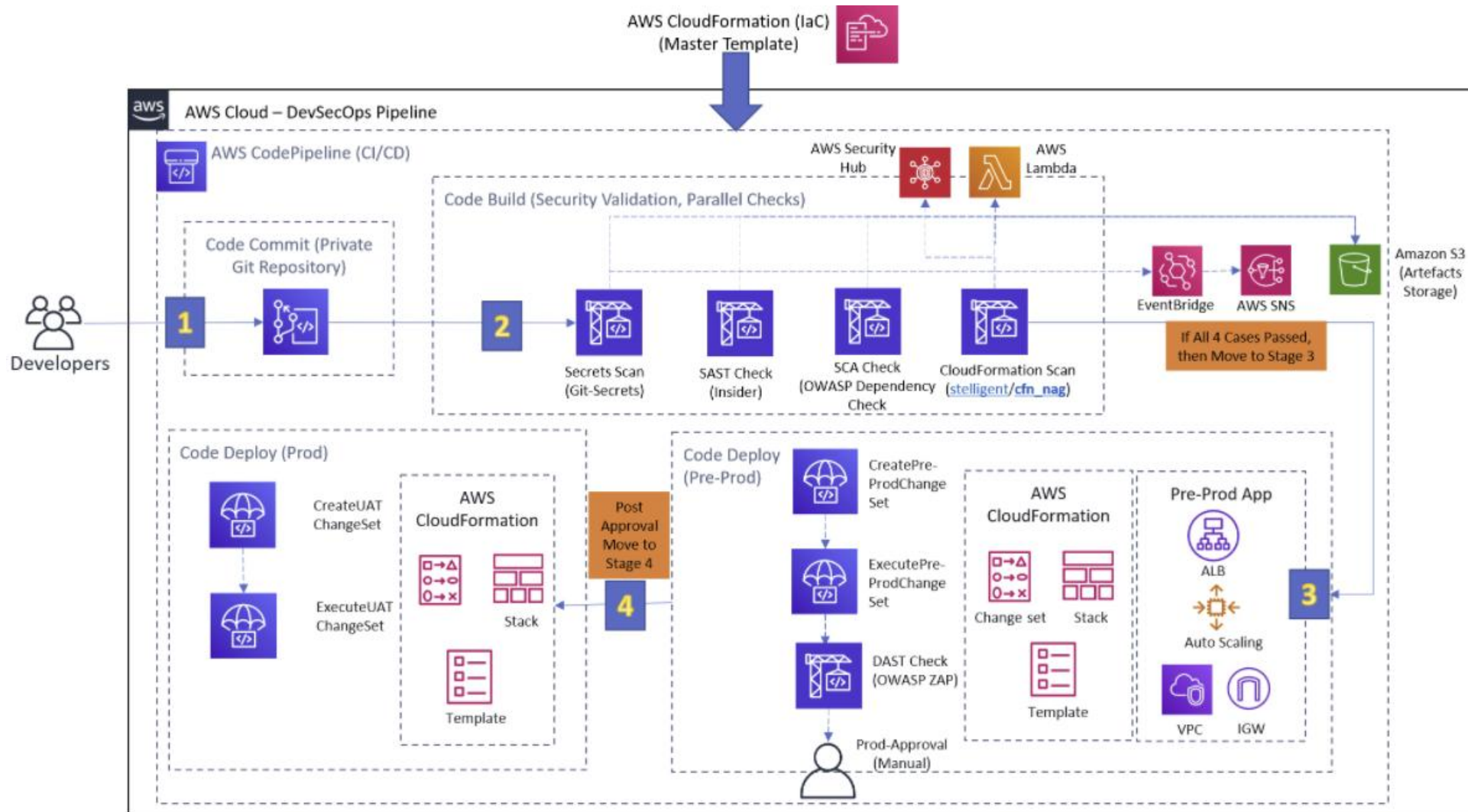
※ 보안약점 (Weakness) : 보안취약점의 근본 원인이 되는 SW 상의 허점, 결점, 오류 등

DevOps환경에서의 DevSecOps



그림출처: 가트너

DevSecOps AWS 아키텍처



DevSecOps AWS 아키텍처

개발자 도구

CodePipeline

▼ 소스 • CodeCommit

시작하기

리포지토리

승인 규칙 템플릿

▶ 빌드 • CodeBuild

▶ 배포 • CodeDeploy

▶ 파이프라인 • CodePipeline

▶ 설정

개발자 도구 > CodePipeline > 파이프라인 > devsecops-workshop-pipeline-pipeline

devsecops-workshop-pipeline-pipeline

알림 ▼ 편집 실행 중지 파이프라인 복제 변경 사항 릴리스

Source 성공
파이프라인 실행 ID: cc27f21e-7267-4801-9a35-2d83bb09027e

SourceAction ⓘ
AWS CodeCommit

성공 - 4시간 전
b4511738

b4511738 SourceAction: exam

전환 비활성화

SCA-and-SAST-Analysis 실패
파이프라인 실행 ID: cc27f21e-7267-4801-9a35-2d83bb09027e 재시도

SCAnalysis ⓘ
AWS CodeBuild

성공 - 4시간 전
세부 정보

SASTAnalysis ⓘ
AWS CodeBuild

실패 - 4시간 전
작업 실행 실패
CodeBuild에서 보기

b4511738 SourceAction: exam

전환 비활성화

DevSecOps AWS 아키텍처

개발자 도구

CodePipeline

▼ 소스 • CodeCommit

시작하기

리포지토리

승인 규칙 템플릿

▶ 빌드 • CodeBuild

▶ 배포 • CodeDeploy

▶ 파이프라인 • CodePipeline

▶ 설정

🟢 **Staging-Deploy-and-DAST-Analysis** 성공
파이프라인 실행 ID: 731a597c-53b3-4d03-b41c-d3fd81eec854

DeployStagingAction ⓘ
AWS Elastic Beanstalk [🔗](#)
🟢 성공 - 7일 전

↓

DASTAnalysis ⓘ
AWS CodeBuild
🟢 성공 - 7일 전
[세부 정보](#)

5d47caad SourceAction: test

↓

전환 비활성화

DevSecOps AWS 아키텍처

개발자 도구

CodePipeline

▼ 소스 • CodeCommit

시작하기

리포지토리

승인 규칙 템플릿

▶ 빌드 • CodeBuild

▶ 배포 • CodeDeploy

▶ 파이프라인 • CodePipeline

▶ 설정

✔ Manual-Approval 성공

파이프라인 실행 ID: 731a597c-53b3-4d03-b41c-d3fd81eec854

ApprovalRequired ⓘ

[수동 승인](#)

✔ 승인됨 - 7일 전

[세부 정보 보기](#)

5d47caad SourceAction: test

↓
전환 비활성화

✔ Production-Deploy 성공

파이프라인 실행 ID: 731a597c-53b3-4d03-b41c-d3fd81eec854

DeployProdAction ⓘ

[AWS Elastic Beanstalk](#)

✔ 성공 - 7일 전

5d47caad SourceAction: test

애플리케이션 보안(2)

보안내재화 Security By Design

안전(Secure)하게 동작되는 제품을 만들기위해 제품의 설계단계에서부터 보안을 고려함으로써 알려진 취약점을 조기에 효과적으로 제거하는 것을 의미한다.

Security By Design – Threat Modeling

위협모델링 프로세스 6단계



1. 각각의 직무 전문가들로 팀 구성



2. 다루어질 위협 환경의 범위 설정



3. 특정 애플리케이션 또는 시스템에 대한 모든 가능 위협 식별



4. 각 위협을 위험 수준에 따라 순위를 매김

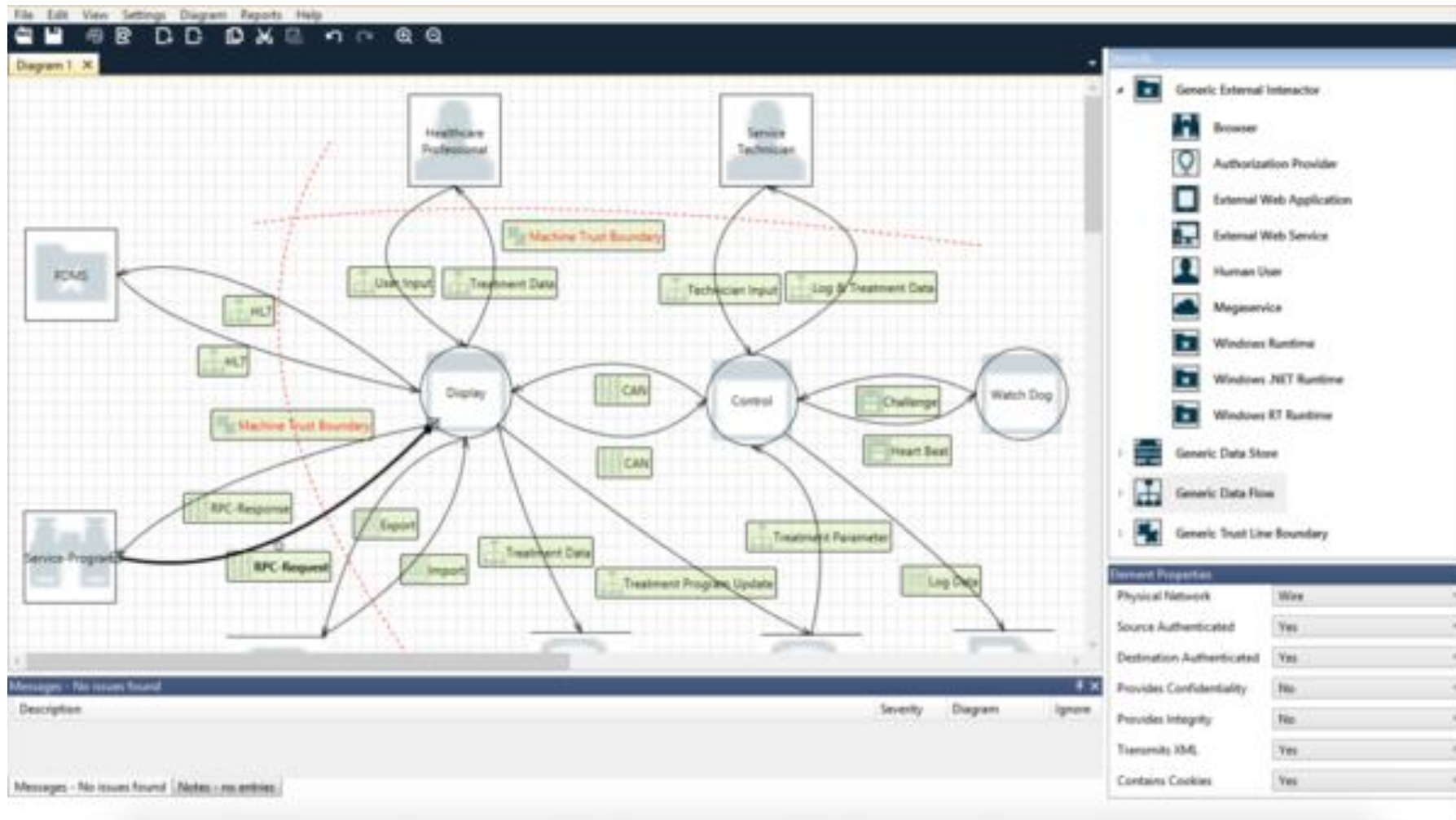


5. 완화 조치를 결정하고 실행



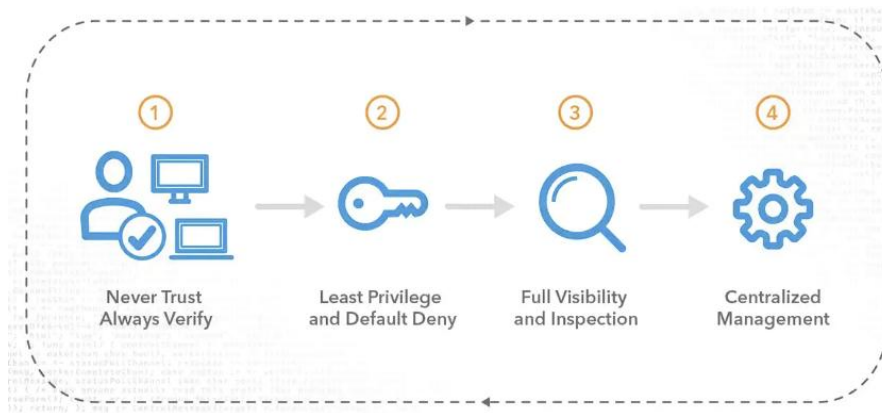
6. 모든 조사 결과와 조치를 문서화

Security By Design – Threat Modeling



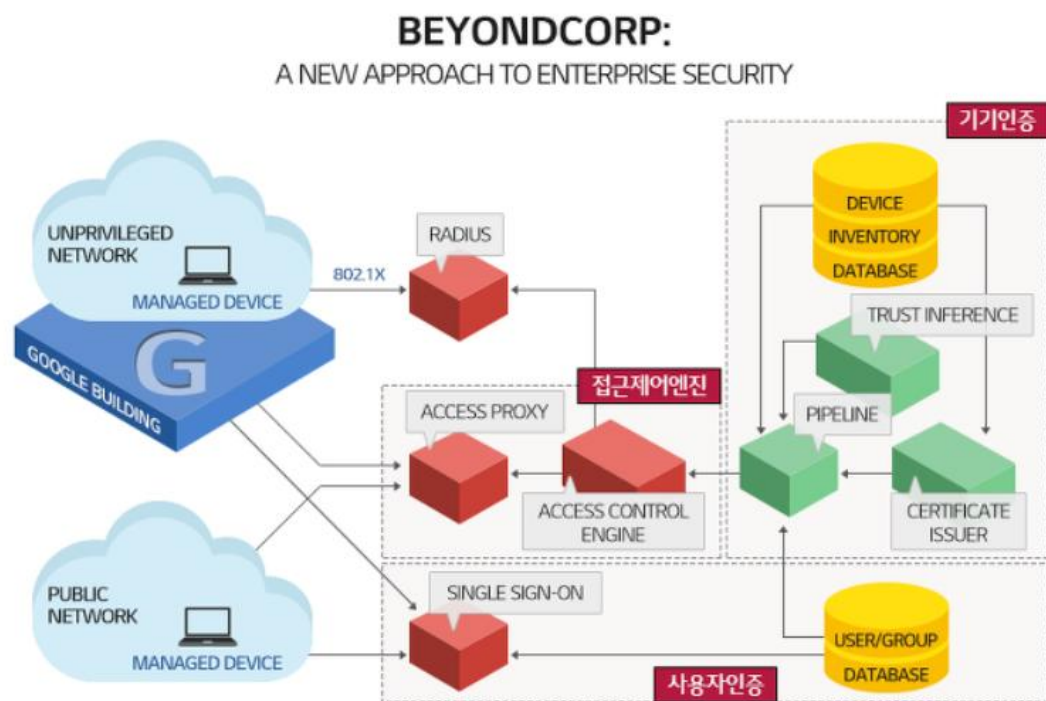
애플리케이션 보안(3)

제로 트러스트 원칙을 사용하여 개발 Zero Trust



제로트러스트는 '아무것도 신뢰할 수 없다는 가정하에, 사용자 및 다양한 정보를 바탕으로 최소한의 권한과 세밀한 통제를 지속적으로 수행하는 보안 활동'
'Never Trust, Always Verify'

제로 트러스트 보안 모델



[BeyondCorp 컴포넌트 1: 기기인증]

접근을 요청하는 호스트가 정상적으로 구글에서 제공한 신뢰할 수 있는 기기임을 확인하는 것으로 인증된 기기에서만 접속 가능

* **디바이스 인벤토리 DB**: 회사 네트워크에 접속하는 모든 디바이스 식별

* **디바이스 아이덴티티**: 관리하는 모든 디바이스는 유일하게 식별, 디바이스 인증서 사용

[BeyondCorp 컴포넌트 2: 사용자 인증]

접근하는 소스트 사용자가 정상적인 구글 임직원인지 확인 및 사용자의 소속 그룹과 업무 확인

* **사용자 인증(User, Group DB)**: 모든 User 및 Group DB와 연동되어 있어 사용자명, 소속그룹, 업무범위, 사용자 근무 위치 정보 반영

* **Single Sign-On System**: 외부에서도 접속 가능한 SSO porta을 통해 이루어 지며 인증상식을 Two-facto로 사용자 계정과 OTP 또는 USB 형태의 물리적 키 사용

[BeyondCorp 컴포넌트 3: 접근제어 엔진(Access Control Engine)]

기기정보, 사용자 정보, 취약점 정보 등을 상관 분석해 사용자 접근 허용 여부를 결정하는 엔진. 접근제어 엔진이 접속 허용 또는 차단 의사결정을 내릴 때 관측데이터와 입력데이터를 사용하여 보안등급평가(Trust Evaluation)과정을 거쳐 반영됨.

* **관측데이터(Observed Data)**: 계획적으로 생성된 데이터(취약점 스캔결과, AD 정책, OS버전 및 패치 여부, 설치된 SW 목록 등)

* **입력데이터(Prescribed Data)**: IT 운영자가 수종으로 관리하는 데이터(기기의 소유자, 기기에 접속할 수 있는 사용자·그룹, DNS, DHCP 정보, 특정 VLAN 등)

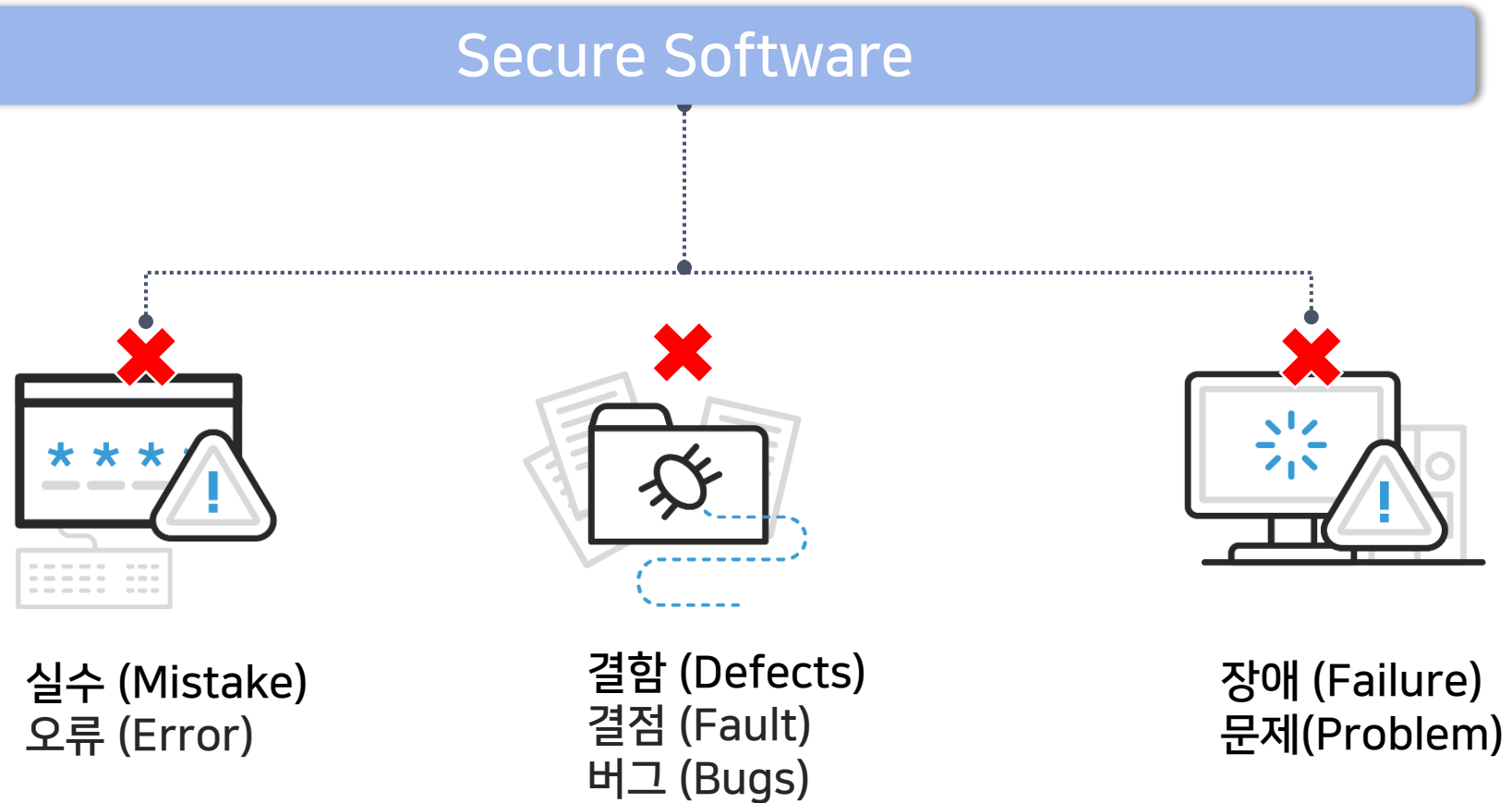


SW개발보안 이해

01 현대 애플리케이션 보안 과제

02 사고사례를 통한 시큐어코딩 이해

시큐어한 소프트웨어란?



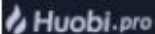
개발자의 사소한 실수 - 시큐어코딩이 적용되지 않은 스마트 컨트랙트

BLOCKCHAIN NEWS

코인 무한복사 가능? SMT 사태로 시장 폭락...후오비선 출금 중단

김동호 기자 | 등록 2018-04-25 20:09:06 | 수정 2018-04-26 10:36:48

SMT토큰 1경 개 이상 복사, 후오비서 이상거래 포착
이더리움 기반 토큰 전부 복사 가능은 사실무근...SMT 스마트컨트랙트 문제

 [법률 통화 거래](#) [통화 거래](#) [레버리지 거래](#) [H20X](#) [로그인](#) [등록](#) [출금/인출](#)

[공지](#) > [공지 사항](#)

Hangco Pro의 모든 통화 인치 통화 서비스 중지

2018-04-25 10:10:13

친애하는 사용자:

SmartMesh (SMT) 프로젝트 아키텍처에서 비정상적인 거래, 예비 조사 후, 허점 SMT 지능형 이더넷 플랫폼 계약이 있는 문제를 발견했다. 현재 통화 프로도 출전 TXID 통화 0x0775e55c402281e8ff24cf37d6f2079bf2a768cf7254593287b5f8a0f621b83 예외 같은 기간에 발견,이 편 돈을 위해 리스크 관리 시스템 요급 차단 (한 <https://etherscan.io/tx/0x0775e55c402281e8ff24cf37d6f2079bf2a768cf7254593287b5f8a0f621b83>).

이로 인해 Firecoin Pro는 모든 통화의 인출 통화 업무를 일시 중지하기로 결정하고 SMT 이상이 확인되면 모든 통화로 통화 인보이스 발행 사업을 재개합니다. 복구 시간은 발표 형식으로 발표됩니다. 정화 기간 중에 발생한 불만을 용서해주시고!

Fire Coin Global Professional Station
2018 년 4 월 25 일

개발자의 사소한 실수 - 시큐어코딩이 적용되지 않은 스마트 컨트랙트

오버플로우

오버플로우란 숫자가 최대치 이상으로 증가하는 것을 말한다.

$$\begin{array}{cccccccc} 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ + & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

개발자의 사소한 실수 - 시큐어코딩이 적용되지 않은 스마트 컨트랙트

문제함수: transferProxy

```
function tranferProxy(address _from, address _to, uint256 _value, uint256 _feeSmt,  
    uint8 _v, bytes32 _r, bytes32 _s) public transferAllowed(_from) return (bool) {  
  
    if(balances[_from] < _feeSmt + _value) revert();  
  
    uint256 nonce = nonces[_from];  
    bytes32 h = keccak256(_from, _to, _value, _feeSmt, nonce);  
    if ( _from != ecrecover(h,_v,_r,_s)) revert();  
  
    if(balance[_to] + value < balances[_to]  
        || balances[msg.sender] + _feeSmt < balances[msg.sender]) revert();  
    balances[_to] += _value;  
    Transfer(_from, _to, _value);  
  
    balances[msg_sender] += _feeSmt;  
  
}
```

개발자의 사소한 실수 - 시큐어코딩이 적용되지 않은 스마트 컨트랙트

세이프매스(SafeMath)

이더리움 스마트컨트랙트 개발에 많이 사용되는 솔리디티(Solidity)에서는 오버플로우(overflow)와 언더플로우(underflow) 처리를 위해 **SafeMath 라이브러리**를 제공하고 있음.
라이브러리는 솔리디티에서 특수한 컨트랙트(Contract)

```
library SafeMath {  
  
    function mul(uint256 a, uint256 b) internal pure returns (uint256) {  
        if (a == 0) {  
            return 0;  
        }  
        uint256 c = a * b;  
        assert(c / a == b);  
        return c;  
    }  
  
    function div(uint256 a, uint256 b) internal pure returns (uint256) {  
        // assert(b > 0); // Solidity automatically throws when dividing by 0  
        uint256 c = a / b;  
        // assert(a == b * c + a % b); // There is no case in which this doesn't hold  
        return c;  
    }  
  
    function sub(uint256 a, uint256 b) internal pure returns (uint256) {  
        assert(b <= a);  
        return a - b;  
    }  
  
    function add(uint256 a, uint256 b) internal pure returns (uint256) {  
        uint256 c = a + b;  
        assert(c >= a);  
        return c;  
    }  
}
```

소프트웨어 보안취약점 사례 - OSS(Open Source Software) 취약점 공격

"방치된 오픈소스 사용"

log4j 교훈 잊었나? 2년 이상 방치된 오픈소스 88%가 사용 중

88%의 조직에서 여전히 최신 오픈소스 업데이트에 미진한 것으로 나타나
오픈소스 라이선스 충돌 및 취약점 감소하는 추세
시놉시스, '2022 오픈소스 보안과 리스크 분석' 보고서 발간

사상 최악의 보안 취약점으로 손꼽히는 Log4j 취약점 발견으로 오픈소스 관리의 중요성이 강조됐지만, 아직도 오픈소스 사용 기업의 88%가 2년 이상 방치된 오픈소스를 사용하는 것으로 알려져 충격을 주고 있다. 시놉시스(Synopsys, Inc.)는 이와 같은 내용이 담긴 전 세계 오픈소스 사용 현황을 분석한 '2022 오픈소스 보안과 리스크 분석(OSSRA)' 연례 보고서를 발표했다. 올해로 7번째 발간된 이번 보고서에는 OSS 라이선스 관리 솔루션인 '블랙덕 오딧 서비스(Black Duck Audit Services)'를 통해 실시한 전 세계 17개 산업분야의 2,400여개의 커머셜 코드 베이스에 대한 분석이 담겨 있다.

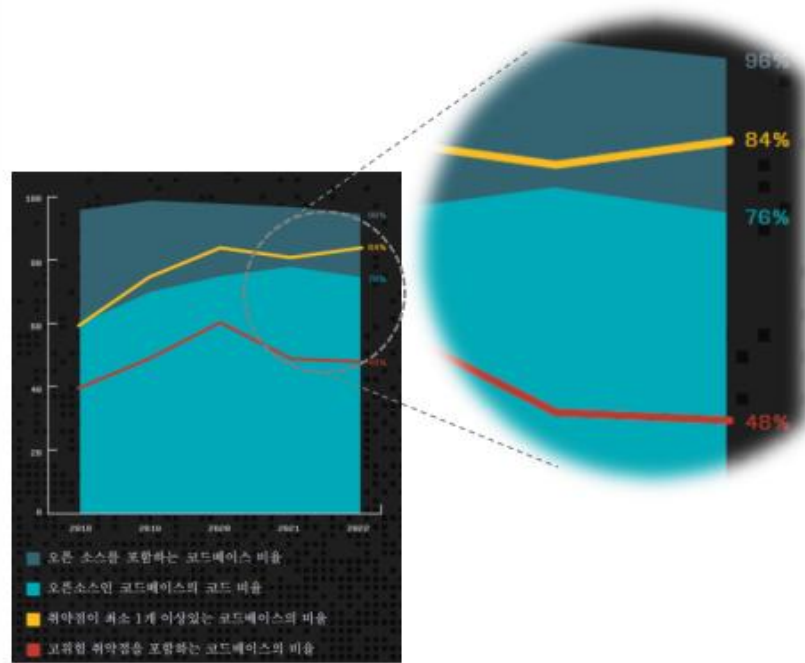
2022 OSSRA 보고서에 따르면 오픈소스가 모든 산업에서 널리 사용되고 있으며 현재 운영되고 있는 대부분의 애플리케이션에 기반을 제공하는 것으로 나타났다. 하지만 최악의 취약점으로 꼽히는 Log4j 등 오래된 오픈소스들이 여전히 표준으로 사용되고 있는 것으로 확인됐다. 운영 리스크/유지보수 측면에서 2,097개의 코드베이스 중 85%가 4년 이상 지난 오픈소스를 포함하고 있는 것으로 나타났다. 88%는 사용 가능한 최신 버전이 아닌 구성 요소를 사용하고 있으며, 이중 5%에 취약한 버전의 Log4j가 포함되어 있다.

"관리를 하지 않아 잊혀지고 나면 사용 위치를 파악하여 업데이트하는데 어려움을 겪게 된다. 최근 Log4j로 촉발된 이슈가 바로 이러한 문제점을 반영하며, 소프트웨어 공급망과 소프트웨어 BOM(원재료 명세서, Bill of Materials)이 중요한 이유가 여기에 있다"고 덧붙였다.

출처: 보안뉴스

"2023 OSSRA 보고서"

상용 소프트웨어의 코드베이스 중 76%는
오픈소스...그 중 48%는 고위험군의 보안 취약점 포함



출처: 2023 OSSRA 보고서

소프트웨어 보안취약점 사례 - OSS(Open Source Software) 취약점 공격

전세계가 두려워했던 Log4j 취약점

- 2021년 11월 24일 Alibaba Cloud 보안팀에 의해 처음 발견
- Java 기반 로깅 도구인 Apache Log4j 라이브러리 원격 코드 실행 취약점
- 공격자가 로그 메시지를 제어할 수 있는 서버에서 로드된 임의의 코드 실행 가능
- Log4j 라이브러리를 사용하는 대부분의 앱이 취약한 것으로 알려짐



해커

① 대상탐색

② 공격코드 전송

웹서버(Apache log4j 2)



Log4j 2 : 자바 기반
로그작성 오픈소스 유틸리티

LOG LOG LOG (로그파일)

예) 접속기록(IP, 시간 등), 로그인 정보
사용자 요청내용, 결과 등



③ Log4j 2 라이브러리 실행중
해커의 공격코드 실행

원격코드실행 취약점
(CVE-2021-44228)

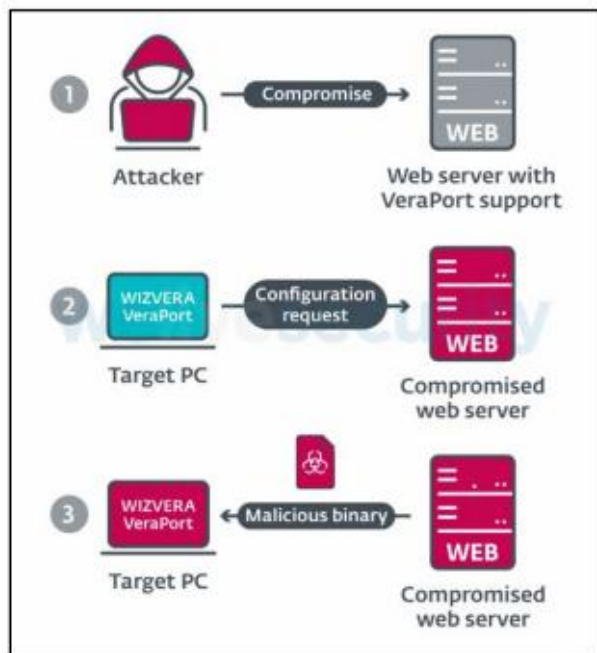
④ 데이터 탈취, 악성코드 다운로드·실행 등

출처: 2023년 SW보안악점 진단원 보수 교육

소프트웨어 보안취약점 사례 - 공급망 해킹

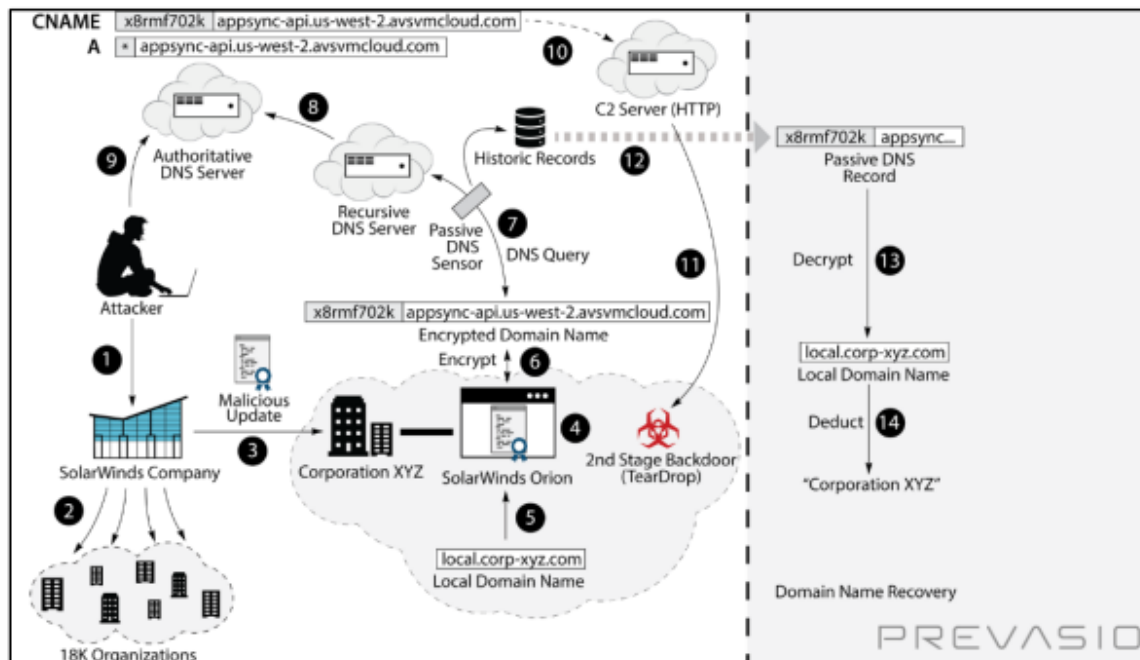
취약한 소프트웨어를 사용하는 기관, 기업등에 피해를 주기 쉬운 공급망 해킹 공격 유형
공격자들은 소프트웨어 개발-설계-구현-유통 단계별 허점 공략

위즈베라 해킹



출처 : Welivesecurity

Solar Winds 해킹



출처 : Prevasio

소프트웨어 보안 취약점 사례 - 공급망 보안

SBOM(Software Bill of Materials)

소프트웨어를 빌드하기 위해 사용된 여러 컴포넌트에 대한 내용과 공급망 관계를 형식에 맞춰 기록하는 것
소프트웨어 제품을 생산, 소비, 운영하는 자가 공급망에 대해 더 깊이 이해하고 알려진
혹은 새로운 취약점과 위협을 추적하여 알려진 취약점의 확산을 효과적으로 저지하기 위한 목적



소프트웨어 개발 주기에 따른 SBOM 정보 (출처 : 한국인터넷진흥원)



요약 정리

- ✓ 현대 애플리케이션의 보안과제에서 API 보안과 지속적인 보안에 대해 학습함.
- ✓ 현대 애플리케이션 보안과제를 해결하기 위한 방안으로 SW개발보안, 보안 내재화, 제로트러스트 보안모델 적용을 학습함.
- ✓ 개발자의 사소한 실수로 발생할 수 있는 보안 사고를 예방하기 위해 시큐어코딩이 필수 보안 활동임을 학습함.



참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [KISA] 클라우드서비스(SaaS)_보안인증기분_해설서(2023.3)
- [KISA] 주요정보통신기반시설 기술적 취약점 분석 평가 상세 가이드(2021.03)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어
- OWASP - <https://owasp.org/>
- CVE - <https://cve.mitre.org/>
- CWE - <https://cwe.mitre.org/>
- SEI CERT Coding Standards - <https://wiki.sei.cmu.edu/confluence/display/seccode>

The background is a vibrant blue gradient. It is decorated with various abstract elements: a large solid blue sphere in the upper left, a smaller sphere with diagonal stripes in the upper right, and another striped sphere in the lower left. Several network-like structures, consisting of small dots connected by thin white lines, are scattered across the image. There are also several horizontal and diagonal bars of different shades of blue. The overall aesthetic is modern and technological.

감사합니다.



퀴즈

1. 다음 중 SW개발보안 프로세스에 해당하지 않는 것은?

1) 설계

2) 요구사항분석

3) 테스트

4) 배포

2. 제품의 설계단계에서부터 보안을 고려하여 알려진 취약점을 조기에 효과적으로 제거하는 것을 의미하는 것은?

1) DevSecOps

2) 보안내재화

3) 제로트러스트

4) 보안일체화

3. 다음 중 '오버플로우'에 대한 설명이 맞는 것은?

1) 연산 결과가 과하게 반올림됨

2) 연산 결과가 허용할 수 있는 범위를 벗어남

3) 연산 결과의 부호가 역전됨

4) 정수형과 실수형을 혼합하여 연산함



퀴즈 정답

1. 다음 중 SW개발보안 프로세스에 해당하지 않는 것은?

4) 배포

SW개발보안 프로세스는 1) 요구사항분석 2)설계 3)구현 4)테스트 로 이루어져 있습니다.

2. 제품의 설계단계에서부터 보안을 고려하여 알려진 취약점을 조기에 효과적으로 제거하는 것을 의미하는 것은?

2) 보안내재화

영어 표현으로 Security By Design이라고 합니다.

3. 다음 중 '오버플로우'에 대한 설명이 맞는 것은?

2) 연산 결과가 허용할 수 있는 범위를 벗어남

저장할 수 있는 값의 범위를 초과하는 연산이 이루어질 때, '오버플로우'가 발생했다고 합니다.